SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1110

COURSE OUTCOME COVERED

CO4:
implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS *(BL5)*

Assessment Tool Weightage: 30%

QUESTIONS: 6 **Total Marks:30**

Annexure A

Reverse Engineering Task

Code of Conduct:

I TEJASWINI M (192411079) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
TEJASWINI M

Annexure A

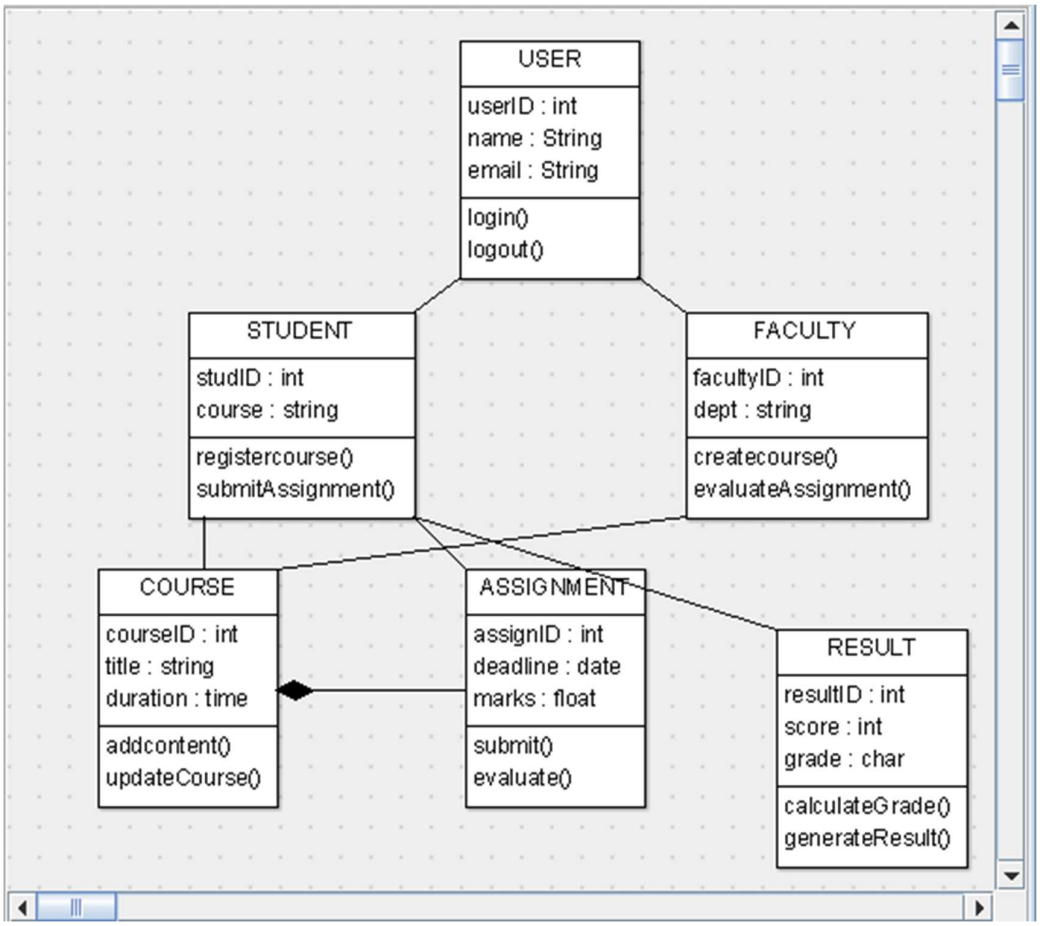
Reverse Engineering Task

1. Legacy E-Learning Management System

An existing **E-Learning Management System** has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using **object-oriented principles and layered architecture**, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.



LAYERED ARCHITECTURE

Presentation Layer

↓

Business Logic Layer

↓

Data Access Layer

↓

Database

- **Presentation Layer:** Login, course and student interfaces.
- **Business Layer:** Registration, course management, evaluation and result processing.
- **Data Access Layer:** Performs database operations.
- **Database:** Stores students, courses, assignments and results.

IMPROVEMENTS

- **High Cohesion:** Each class/module performs a specific responsibility.
- **Low Coupling:** UI, business logic and database are separated.
- **Maintainability:** Changes in one layer have minimal effect on others.
- **Scalability:** New modules such as online exams or certificates can be added easily.
- **Reusability:** Common User functionality can be reused by Student and Faculty.

CONCLUSION

The redesigned system replaces the tightly coupled legacy structure with object-oriented classes and layered architecture. This provides better cohesion, lower coupling, easier maintenance, greater reusability, and improved scalability.

2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable **behavioral design patterns**, and explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

EXISTING SYSTEM – PROBLEMS
The old system uses many if-else / switch conditions:
if channel = Email
else if channel = Chatbot
else if channel = Telephone
else if channel = SocialMedia

LIMITATIONS

- High coupling
- Difficult to add new channels
- Changes affect existing modules
- Poor maintainability
- Violates the Open/Closed Principle

RECOMMENDED BEHAVIORAL DESIGN PATTERN
Strategy Pattern
The Strategy Pattern is most suitable because each support channel can be treated as a different strategy.

Classes
SupportChannel *(Interface)*

- processRequest()

EmailSupport

- processRequest()

ChatbotSupport

- processRequest()

TelephoneSupport

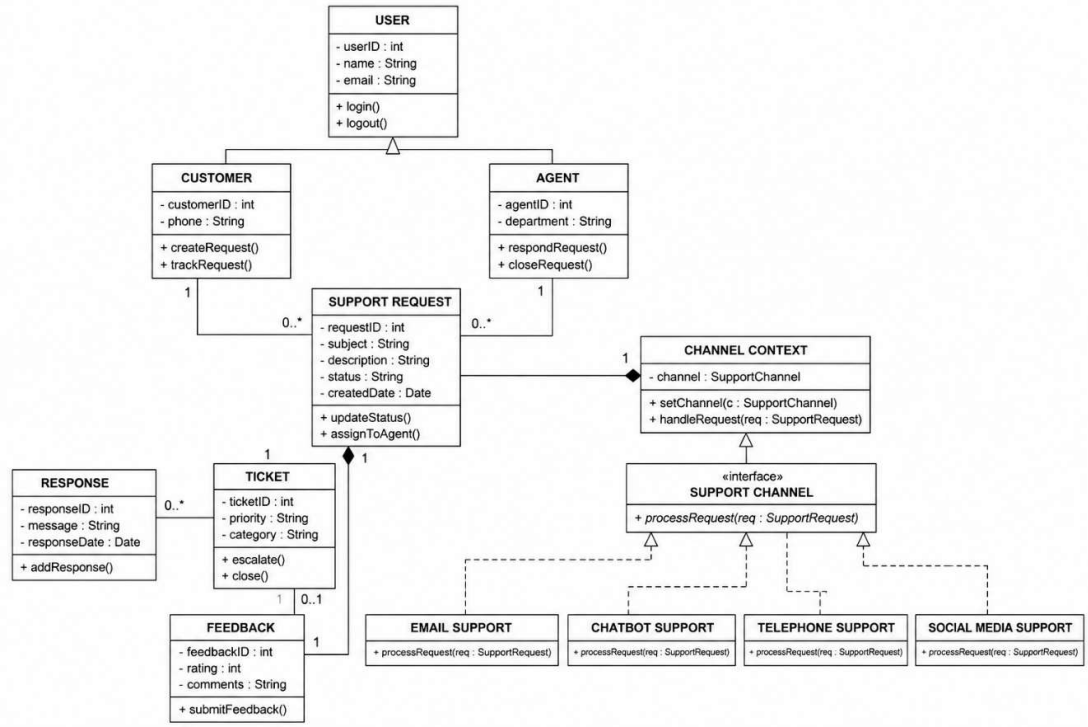
- processRequest()

SocialMediaSupport

- processRequest()

SupportManager

- handleRequest()



HOW STRATEGY PATTERN WORKS

Instead of checking the channel using multiple conditions, SupportManager uses the selected SupportChannel.

For example:

Customer Request

↓

SupportManager

↓

Selected SupportChannel

↓

Email / Chatbot / Telephone / Social Media

If a new WhatsAppSupport channel is required, simply create a new class implementing SupportChannel.

IMPROVEMENTS

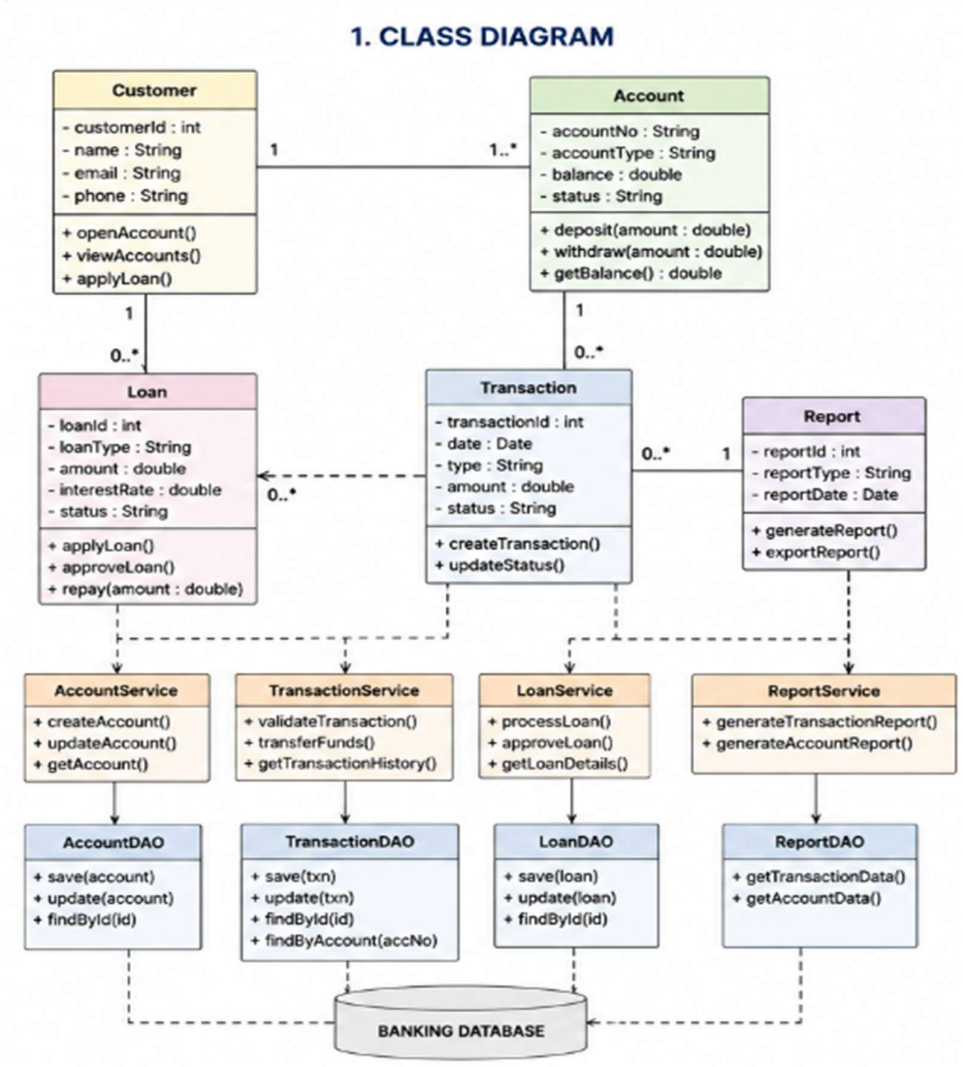
Aspect	**Improvement**
Flexibility	Channel can be changed dynamically
Extensibility	New channels can be added easily
Loose Coupling	Manager depends on interface, not concrete channels
Maintainability	Each channel has separate implementation
Reusability	Support strategies can be reused

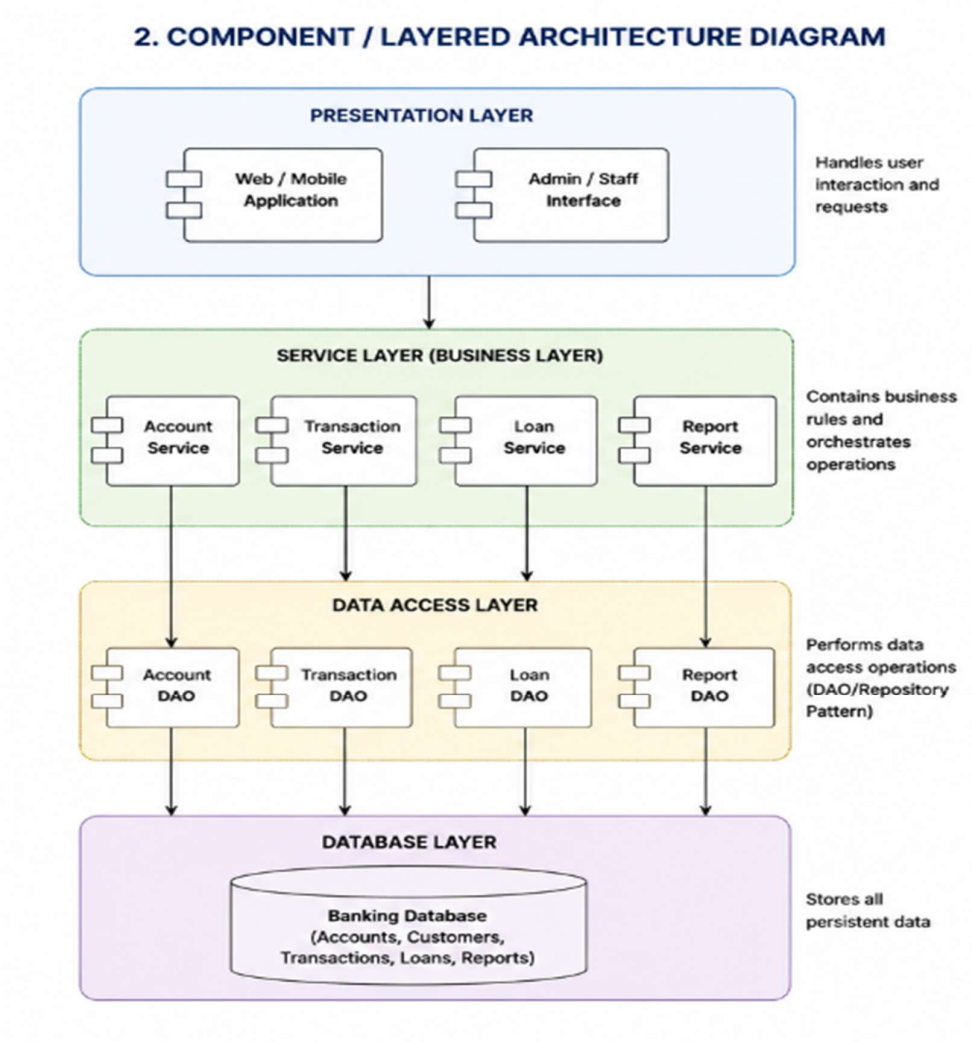
3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate **domain classes, service classes, and data access components**. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.





DESIGN PATTERN

Repository/DAO Pattern

Use DAO/Repository classes to separate database operations from business logic.

Example:

TransactionService → TransactionDAO → Database

So TransactionService does not directly execute database operations.

Service Layer Pattern

Business operations are placed inside service classes:

TransactionService → validateTransaction(), transferFunds()

HOW THE REDESIGN IMPROVES THE SYSTEM

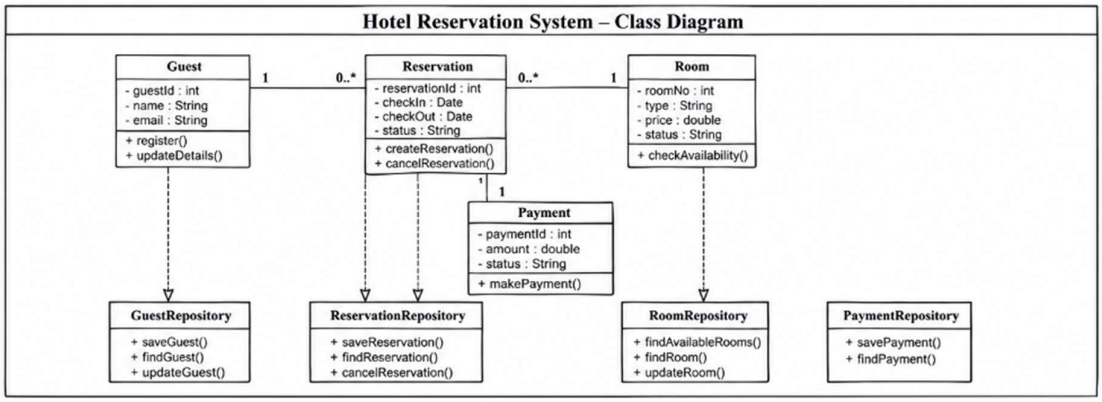
Aspect	**Improvement**
Separation of concerns	UI, business logic and database are separated
Testability	Services can be tested independently
Maintainability	Database changes do not affect business classes
Loose coupling	Classes communicate through defined interfaces
Scalability	New services and banking features can be added easily

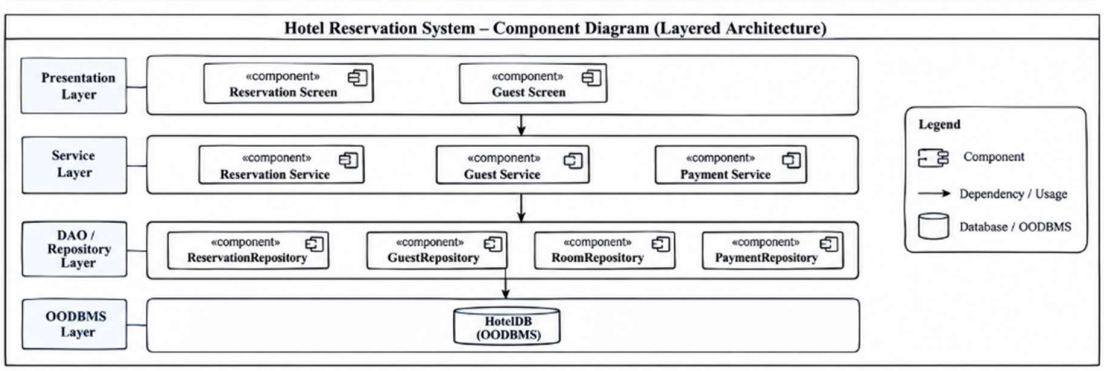
4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to **coupling, duplication, and persistence management**. Reverse engineer the system by introducing suitable **DAO, Repository, or Data Access patterns**, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.





PROBLEMS IN EXISTING SYSTEM

High Coupling

Reservation screens directly communicate with the OODBMS.

Duplication

The same room, guest, booking, and payment queries are repeated in different modules.

Poor Persistence Management

Database operations are mixed with business logic.

HOW THE REDESIGN HELPS

- **Modularity:** UI, business logic, and persistence are separated.
- **Reusability:** Common database operations are placed in repositories.
- **Maintainability:** Database changes can be handled mainly in the DAO/Repository layer.
- **Loose Coupling:** Screens no longer directly depend on the OODBMS.
- **OODBMS Integration:** Repository classes provide a single controlled interface for persistent objects.

Best Pattern

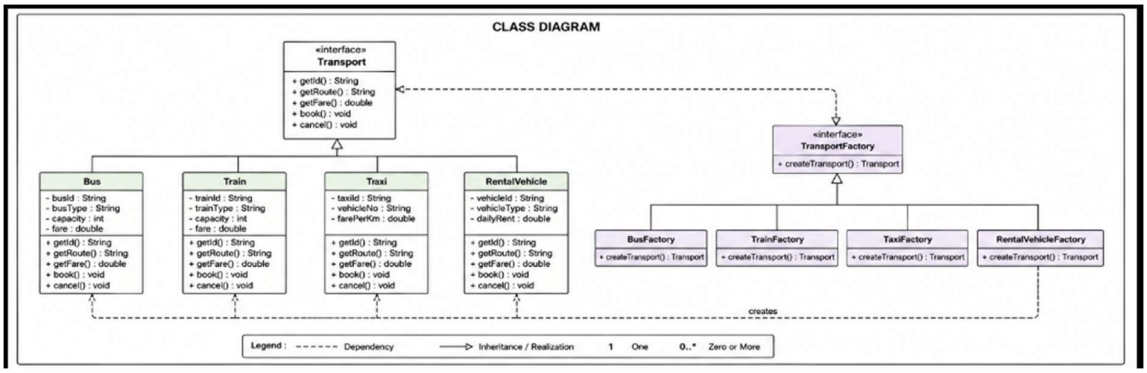
Use the Repository/DAO Pattern to centralize persistence operations.

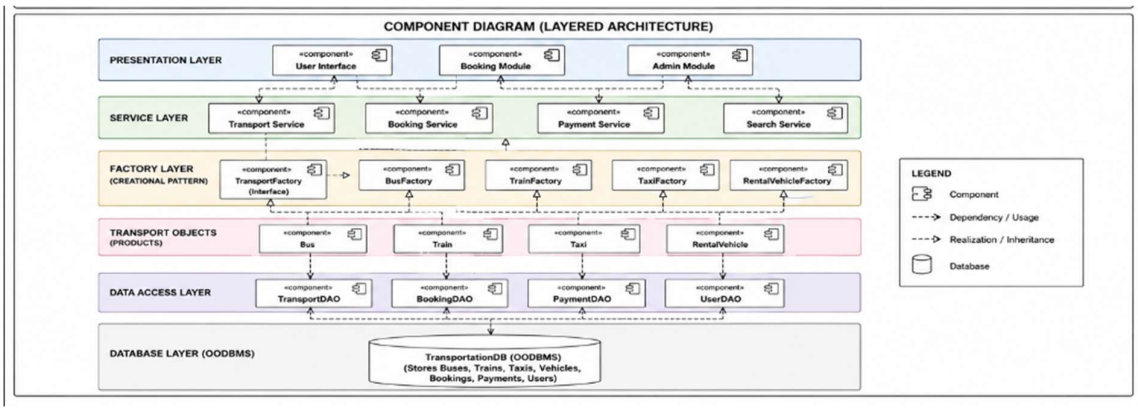
5.Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable **creational design patterns**, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.





MAIN FLOW
User Interface → Transport Service → TransportFactory → Bus/Train/Taxi/RentalVehicle

and for storage:
Service Layer → DAO → TransportationDB

DESIGN PATTERN
Factory Method Pattern is the main pattern used because it centralizes object creation. If a new type such as Metro is introduced, a new Metro class and MetroFactory can be added without modifying existing transport classes.

BENEFITS

- **Extensibility:** New transport types can be added easily.
- **Scalability:** System can support more transportation services.
- **Code Reuse:** Common operations are defined in Transport.
- **Loose Coupling:** Clients depend on the Transport abstraction rather than concrete classes.
- **Maintainability:** Object creation is separated from business logic.

6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of **SOLID principles**, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

SOLID VIOLATIONS IN LEGACY SYSTEM

SRP – Single Responsibility Principle

Problem: One class performs sensor reading, alert generation, database storage and report generation.

Solution: Separate them into Sensor, AlertService, RecordRepository, and ReportService.

OCP – Open/Closed Principle

Problem: Adding a new sensor requires modifying existing monitoring code.

Solution: Create the Sensor interface. New sensors can implement Sensor without changing existing classes.

DIP – Dependency Inversion Principle

Problem: Services directly depend on concrete monitoring devices and databases.

Solution: Services depend on abstractions such as Sensor and RecordRepository.

DESIGN PATTERN

Strategy Pattern

Use the **Strategy Pattern** for different sensor types.

Sensoe

├── HeartRateSensor

├── TemperatureSensor

└── BloodPressureSensor

A new sensor can be added without modifying the existing monitoring system.

Repository Pattern

Use the **Repository Pattern** for medical-record storage.

ReportService

↓

RecordRepository

↓

DatabaseRepository

↓

Database

This keeps database operations separate from business logic.

BENEFITS OF REDESIGN

Principle	**Improvement**
SRP	Each class has one clear responsibility
OCP	New sensors can be added without changing existing code
DIP	Services depend on interfaces instead of concrete classes
Cohesion	Related functionality stays within one class
Loose Coupling	Components can change independently
Testability	Interfaces can be replaced with mock implementations
Maintainability	Changes are easier and localized

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation